BEE 4750 Lab 3: Mixed Integer Programming and Facility Siting in JuMP

2026-11-09

> Due Date
>
> Wednesday, November 11, 9:00pm

Setup

Run these two cells first. The first time, installing and loading `JuMP` and `HiGHS` can take a few minutes.

```
import Pkg
Pkg.activate(".")
Pkg.instantiate()
```

```
using JuMP    # optimization modeling syntax
using HiGHS   # the solver
```

Introduction

Indicator variables can model decisions that are made or not made: whether to operate a plant or whether a facility is built or not built. In this lab you use mixed integer linear programming to decide where a county should build composting sites. The formulation is given below: your job is to implement it in `JuMP`, solve it with and without the indicator variables, and work out how the indicators change based on additional constraints.

The County

A county's four towns collect organic matter (food scraps and yard waste) at the curb every week. The county wants to compost all of it and has three candidate sites for composting facilities. Site A is next to Town 2, the largest town; Site B is in the east, near Town 3; and Site C is in the south, near Town 4.

- Each site can compost up to its **capacity** each week.
- Each site that is built has a **fixed cost**: building it (spread over its working life) and staffing and running it, whether it composts one ton a week or its full capacity.
- Hauling organic matter from a town to a site has a **transport cost** per ton, which grows with the distance.

We assume composting costs the same per ton at every site, so we leave that cost out: it would add the same amount to every plan.

Town	Organic matter (tons/week)	To Site A ($/ton)	To Site B ($/ton)	To Site C ($/ton)
Town 1	60	5	17	22
Town 2	140	4	10	18
Town 3	80	12	6	22
Town 4	100	22	20	6

Site	Capacity (tons/week)	Fixed cost ($/week)
A	300	2,800
B	200	2,200
C	250	2,600

The code below holds the same data, with sites A, B and C as sites 1, 2 and 3.

```
town_names = ["Town 1", "Town 2", "Town 3", "Town 4"]
site_names = ["A", "B", "C"]
n_towns = length(town_names)
n_sites = length(site_names)

organic_matter = [60, 140, 80, 100]  # collected by each town [tons/week]
capacity = [300, 200, 250]           # most a site can take [tons/week]
fixed_cost = [2800, 2200, 2600]      # if the site is built [$/week]

# transport_cost[i, j]: hauling one ton from town i to site j [$/ton]
```

```
transport_cost = [5 17 22;
                  4 10 18;
                  12 6 22;
                  22 20 6]
```

The Model

The decision variables are

- $x_{ij} \geq 0$, the organic matter hauled from town i to site j each week (tons/week);
- $y_j \in \{0, 1\}$, which is 1 if site j is built and 0 if it is not.

With d_i the organic matter town i collects, K_j and F_j the capacity and fixed cost of site j, and c_{ij} the transport cost from town i to site j, the decision problem is

$$\begin{aligned} \min_{\mathbf{x},\mathbf{y}} \quad & \sum_i \sum_j c_{ij} x_{ij} + \sum_j F_j y_j \\ \text{subject to:} \quad & \sum_j x_{ij} = d_i \qquad \text{for each town } i \\ & \sum_i x_{ij} \leq K_j y_j \qquad \text{for each site } j \\ & x_{ij} \geq 0, \quad y_j \in \{0, 1\}. \end{aligned}$$

The first set of constraints says that all of each town's organic matter goes to some site. The second is a constraint enforcing that a site that is built ($y_j = 1$) takes at most its capacity, and a site that is not built ($y_j = 0$) takes nothing.

> **Two Ways to Work Through This Lab**
>
> You can do the coding either in this notebook (`lab03.ipynb`) or in the plain script `lab03.jl`, whichever you prefer: both contain the same steps. Either way, the written parts go on the separate **Activity Sheet** (handed out in class), which also repeats the data and the model. Submit your completed Activity Sheet to Gradescope as a PDF by 9:00pm on Wednesday, November 11; you don't need to submit your code.

> **JuMP Syntax for This Lab**
>
> To declare an array of continuous decision variables, or an array of indicator decision variables:

```
# a table of variables, amount[i, j]
@variable(model, amount[1:n_rows, 1:n_cols] >= 0)
# indicator variables: each is 0 or 1
@variable(model, chosen[1:n_cols], Bin)
```

This is an example of how to declare an array of constraints:

```
@objective(model, Min,
    sum(cost[i, j] * amount[i, j] for i in 1:n_rows, j in 1:n_cols))
@constraint(model, [j in 1:n_cols],
    sum(amount[i, j] for i in 1:n_rows) <= limit[j])
```

To name an array of constraints, put the name before the index, as in `@constraint(model, within_limit[j in 1:n_cols], ...)`.
Put a whole model in one cell and re-run that whole cell after any change: running an `@variable` line again in the same model gives an error, and running an `@constraint` line again adds a second copy of the constraint.

Step 1: Build Every Site

Suppose the county builds all three sites. Then every $y_j = 1$: the fixed costs are a constant, the capacity constraints become plain capacity limits, and the problem is a linear program,

$$\begin{aligned} \min_{\mathbf{x}} \quad & \sum_i \sum_j c_{ij} x_{ij} + (F_A + F_B + F_C) \\ \text{subject to:} \quad & \sum_j x_{ij} = d_i \qquad \text{for each town } i \\ & \sum_i x_{ij} \le K_j \qquad \text{for each site } j \\ & x_{ij} \ge 0. \end{aligned}$$

Code and solve the model below. In the code, `shipped[i, j]` is x_{ij}.

```
all_built = Model(HiGHS.Optimizer)
set_silent(all_built)

# Organic matter hauled from town i to site j [tons/week]
@variable(all_built, shipped[1:n_towns, 1:n_sites] >= 0)

# TODO: the objective, the total cost [$/week]: the transport cost of every
```

```
# shipment, plus the fixed costs of all three sites
# @objective(all_built, Min, ...)

# TODO: one constraint per town: all its organic matter goes to some site
# @constraint(all_built, [i in 1:n_towns], ...)

# TODO: one constraint per site: it takes no more than its capacity
# @constraint(all_built, [j in 1:n_sites], ...)

print(all_built)
```

Check the printed model against the LP above, then solve it. The status should be `OPTIMAL`.

```
optimize!(all_built)
termination_status(all_built)
```

```
@show objective_value(all_built);
display(value.(shipped))
```

If your model is right, the total cost is $9,540 per week. If you get a different number, compare `print(all_built)` with the LP above.

Record the plan in the tables on the Activity Sheet.

Step 2: Let the Model Choose the Sites

Now the county builds only the sites that are worth their fixed cost to operate: each y_j is an unknown indicator decision variable, and the model is the mixed integer linear program in The Model section above. Code and solve the model below. In the code, `site_open[j]` is y_j.

```
siting = Model(HiGHS.Optimizer)
set_silent(siting)

# Organic matter hauled from town i to site j [tons/week]
@variable(siting, shipped[1:n_towns, 1:n_sites] >= 0)
# 1 if the site is built, 0 if not
@variable(siting, site_open[1:n_sites], Bin)

# TODO: the objective, the total cost [$/week]: the transport cost of
# every shipment, plus the fixed cost of each site that is built
```

```
# @objective(siting, Min, ...)

# TODO: one constraint per town: all its organic matter goes to some site
# @constraint(siting, [i in 1:n_towns], ...)

# TODO: one capacity constraint per site: it takes no more than its
# capacity times site_open
# @constraint(siting, [j in 1:n_sites], ...)

print(siting)
```

```
optimize!(siting)
termination_status(siting)
```

As in Step 1, query the total cost and the shipments, and also the values of `site_open`.

```
# TODO: query the solution
```

This model's variables are also called `shipped`, so `value.(shipped)` now gives this model's shipments, not Step 1's. The solver may print values like `-0.0` or `-5.68434e-14` for 0: these are rounding artifacts. Record the plan on the Activity Sheet and answer the Step 2 questions there.

Step 3: A Watershed Rule

Sites A and C both drain to the same creek, a few miles upstream of a drinking-water intake. Runoff from compost piles carries nutrients, so the state will permit **at most one composting site in that creek's watershed**.

Write this rule as a constraint on the y_j (Step 3a on the Activity Sheet). Then build the Step 2 model again below as a new model, `with_rule`: copy in your Step 2 objective and constraints, changing `siting` to `with_rule`, and add the rule. Name the capacity constraints and the rule as the comments say, for Step 4. Remember that sites A, B and C are 1, 2 and 3 in the code.

```
with_rule = Model(HiGHS.Optimizer)
set_silent(with_rule)

# Organic matter hauled from town i to site j [tons/week]
@variable(with_rule, shipped[1:n_towns, 1:n_sites] >= 0)
# 1 if the site is built, 0 if not
```

```
@variable(with_rule, site_open[1:n_sites], Bin)

# TODO: the objective and constraints from Step 2, on with_rule. Name the
# capacity constraints capacity_limit, for Step 4:
# @constraint(with_rule, capacity_limit[j in 1:n_sites], ...)

# TODO: the watershed rule, named watershed: at most one of Sites A and C
# @constraint(with_rule, watershed, ...)

print(with_rule)
```

```
optimize!(with_rule)
termination_status(with_rule)
```

Query the new plan as in Step 2.

```
# TODO: query the solution
```

Record the plan on the Activity Sheet and answer the Step 3 question there.

Step 4: Shadow Prices

A mixed integer program has no shadow prices of its own. `fix_discrete_variables(model)` fixes each indicator variable at its optimal value, so that solving again gives an LP with the same solution. Its shadow prices are **conditional** on that choice: they describe small changes to the outcome assuming the indicator variables have the same values as in the optimal solution.

Fix the indicator variables of the Step 3 model and solve it again.

```
fix_discrete_variables(with_rule)
optimize!(with_rule)
termination_status(with_rule)
```

Then query the shadow prices of the three capacity constraints and of the watershed rule. `shadow_price.` works on an array of constraints as `value.` does on an array of variables.

```
# TODO: query the shadow prices
```

Record them on the Activity Sheet and answer the Step 4 questions there.